



Cryptographic Action Authorization: Capability Token Binding Protocol

Public Technical Disclosure – Prior Art Publication

Publication Date: January 8, 2026

License: CC0 1.0 Universal (Public Domain Dedication)

Status: Defensive Publication for Prior Art

Abstract

This document discloses a cryptographic token format and verification protocol that binds authorization decisions to specific actions, policies, and temporal-subject constraints. Capability tokens create an **unbreakable link** between what an autonomous agent proposes to do and what it is permitted to do, preventing unauthorized action execution even when orchestration layers are compromised.

This specification is published as prior art to establish the standard capability token binding pattern for AI agent authorization systems and prevent patent enclosure of cryptographic action authorization methods.

1. Problem Statement

Autonomous agents (AI systems, bots, automation tools) increasingly execute actions on behalf of users. A critical security requirement exists:

Only actions explicitly authorized by a policy engine can execute, and the token proving authorization must be cryptographically bound to the specific action.

Without cryptographic binding:

- Tokens become generic permissions with no action specificity
 - Attackers can redirect authorized intents to unauthorized targets
 - Policy decisions become meaningless
 - Audit trails cannot prove what was authorized vs. what executed
-

2. Capability Token: Core Structure

A capability token is a signed authorization artifact containing:^{[1][2]}

```
{
  "token_id": "cap_0d2b...",
  "issued_at": "2026-01-08T00:00:01Z",
  "expires_at": "2026-01-08T00:05:01Z",
  "issuer": {
    "issuer_id": "policy_engine_v1",
    "key_id": "ed25519_k1"
  },
  "subject": {
    "subject_id": "usr_123",
    "session_id": "sess_abc",
    "client_pubkey": "BASE64URL..."
  },
  "audience": {
    "tool": "email",
    "adapter_id": "email_adapter_v1"
  },
  "binding": {
    "policy_bundle_hash": "sha256:...",

```

```

    "intent_hash": "sha256:...",
    "params_binding": "HASH_ONLY"
  },
  "scope": {
    "actions": ["send"],
    "constraints": {
      "max_recipients": 5,
      "no_external_domains": false
    }
  },
  "limits": {
    "rate_limit": {
      "per_minute": 3
    },
    "budget": {
      "period": "P1D",
      "asset": "credits",
      "max_amount": 100
    }
  },
  "replay_protection": {
    "nonce": "n_01H...",
    "one_time": true
  },
  "proof": {
    "proof_level": "ATTESTATION",
    "evidence_refs": ["ev_mfa_045", "ev_beneficiary_008"]
  },
  "signature": "BASE64URL..."
}

```

3. Binding Invariants: The Four Security Properties

A valid capability token must satisfy four mathematical invariants:^{[2][1]}

3.1 Invariant 1: Identity Binding

```
token.subject_id_hash = Poseidon(subject_secret, salt)
```

Property: The subject cannot be changed without invalidating the token. The token is non-transferable by design.

Why it matters: A stolen token cannot be used by an attacker under their own identity.

3.2 Invariant 2: Intent Binding

```
token.intent_hash = SHA256(canonicalize(intent_object))
```

Property: The token authorizes exactly one intent. Parameter changes require reissuance.

Why it matters: An attacker cannot redirect an authorized email send from "alice@example.com" to "attacker@example.com"—the intent hash would not match.

3.3 Invariant 3: Policy Binding

```
token.policy_bundle_hash = Hash(policy_rules, version, issuedat)
```

Property: The token is bound to a specific version of policy rules. Policy updates invalidate previously issued tokens.

Why it matters: If policy changes to restrict an action, older tokens cannot be replayed to bypass the new rules.

3.4 Invariant 4: Temporal Validity

```
token.not_before ≤ current_time ≤ token.expires_at ± clock_skew
```

Property: Tokens are time-bounded. Expiry is passive (no on-chain transaction required). Clock skew tolerance is configurable (default 30 seconds).

Why it matters: Tokens become worthless after expiry, preventing indefinite use if stolen.

4. The Three Types of Parameter Binding

Capability tokens support three parameter binding patterns:^[1]

4.1 HASH_ONLY Binding (Default)

Token binds to `intent_hash`, which implicitly binds all parameters via canonicalization:

Token validates:

1. Reconstruct canonical form of intent
2. Compute `intent_hash = SHA256(canonical_bytes)`
3. Compare with `token.binding.intent_hash`
4. If mismatch → Token invalid (tampering detected)

Advantage: Flexible, works with sealed/encrypted parameters.

Use case: Email, web navigation where parameters are large or sensitive.

4.2 CONSTRAINT Binding (Explicit Bounds)

Token includes explicit constraints that adapters verify at runtime:

```
{
  "constraints": {
    "amount_max": "100.00",
    "beneficiary_id": "ben_001",
    "asset": "USDC"
  }
}
```

Adapter verifies:

```
if (requested_amount > constraints.amount_max) → DENY
if (requested_beneficiary != constraints.beneficiary_id) → DENY
```

Advantage: Transparent, auditable at runtime.

Use case: Payments, database queries where constraints are enumerated.

4.3 SEALED_PARAMS Binding (Reference-Based)

Sensitive parameters stored in sealed storage; token references by commitment only:

Token contains:

```
token.params_commitment = Hash(sensitive_params)
```

Adapter flow:

1. Receive token
2. Verify token signature and bindings
3. ONLY AFTER token verification, resolve actual params by reference
4. Execute action with resolved params

Advantage: Maximum privacy—policy engine never sees sensitive data.

Use case: Healthcare (PII), finance (accounts), database (credentials).

5. Verification Protocol

A tool adapter MUST verify tokens before executing any action:^[1]

5.1 Verification Checklist (Normative)

The adapter MUST perform these checks in order:

1. SIGNATURE VERIFICATION

- ✓ Token signature is valid under issuer's key
- ✓ Key ID (kid) matches expected issuer
- ✓ Signature algorithm is acceptable (e.g., Ed25519)

2. TEMPORAL VALIDITY

- ✓ $\text{Token.nbf (not-before)} \leq \text{current_time} + \text{skew}$
- ✓ $\text{Token.exp (expires)} \geq \text{current_time} - \text{skew}$
- ✓ Skew tolerance is configurable, default 30 seconds

3. AUDIENCE BINDING

- ✓ $\text{Token.audience.tool}$ matches this tool (e.g., "email")
- ✓ $\text{Token.audience.adapter_id}$ matches this adapter (e.g., "email_adapter_v1")

4. SUBJECT & SESSION BINDING

- ✓ $\text{Token.subject.subject_id}$ matches requesting subject
- ✓ $\text{Token.subject.session_id}$ matches current session (if stateful)
- ✓ If client_pubkey present, verify session key binding

5. INTENT HASH VERIFICATION

- ✓ Reconstruct canonical intent from request
- ✓ Compute `intent_hash = SHA256(canonical_bytes)`
- ✓ Compare with `token.binding.intent_hash`
- ✓ If mismatch → BLOCK action (tampering detected)

6. SCOPE ENFORCEMENT

- ✓ Requested action $\in$ `token.scope.actions`
- ✓ All requested params satisfy `token.scope.constraints`
- ✓ Parameter values are within bounds

7. REPLAY PROTECTION

- ✓ If `token.replay_protection.one_time = true`:
 - Verify nonce has not been used before
 - Atomically mark nonce as used
 - Reject if already in use cache
- ✓ Stateless nullifiers may be used instead of state

8. BUDGET/RATE LIMIT ENFORCEMENT

- ✓ Do NOT trust upstream budget tracking
- ✓ Locally enforce limits defined in `token.limits`
- ✓ Decrement budget atomically per execution
- ✓ Block if budget exhausted

9. REVOCATION CHECK (Optional but Recommended)

- ✓ Verify token is not in active revocation list
- ✓ Revocation may be explicit or implicit (policy update)

5.2 Execution Flow Diagram

Request arrives at adapter:

↓

[^1] Verify signature valid?

NO → BLOCK, return 401 INVALID_SIGNATURE

YES ↓

[^2] Token not expired?

NO → BLOCK, return 401 EXPIRED

YES ↓

```
[^3] Audience matches?
    NO → BLOCK, return 401 WRONG_AUDIENCE
    YES ↓
[^4] Subject & session match?
    NO → BLOCK, return 401 UNAUTHORIZED
    YES ↓
[^5] Intent hash matches request?
    NO → BLOCK, return 403 TAMPERING_DETECTED
    YES ↓
[^6] Scope constraints satisfied?
    NO → BLOCK, return 403 FORBIDDEN
    YES ↓
[^7] Nonce not replayed (if one_time)?
    NO → BLOCK, return 403 ALREADY_USED
    YES ↓
[^8] Budget not exhausted?
    NO → BLOCK, return 429 BUDGET_EXHAUSTED
    YES ↓
[^9] Not revoked?
    NO → BLOCK, return 403 REVOKED
    YES ↓
EXECUTE action
    ↓
Emit audit event
    ↓
Return receipt
```

6. Token Issuance (Policy Engine Side)

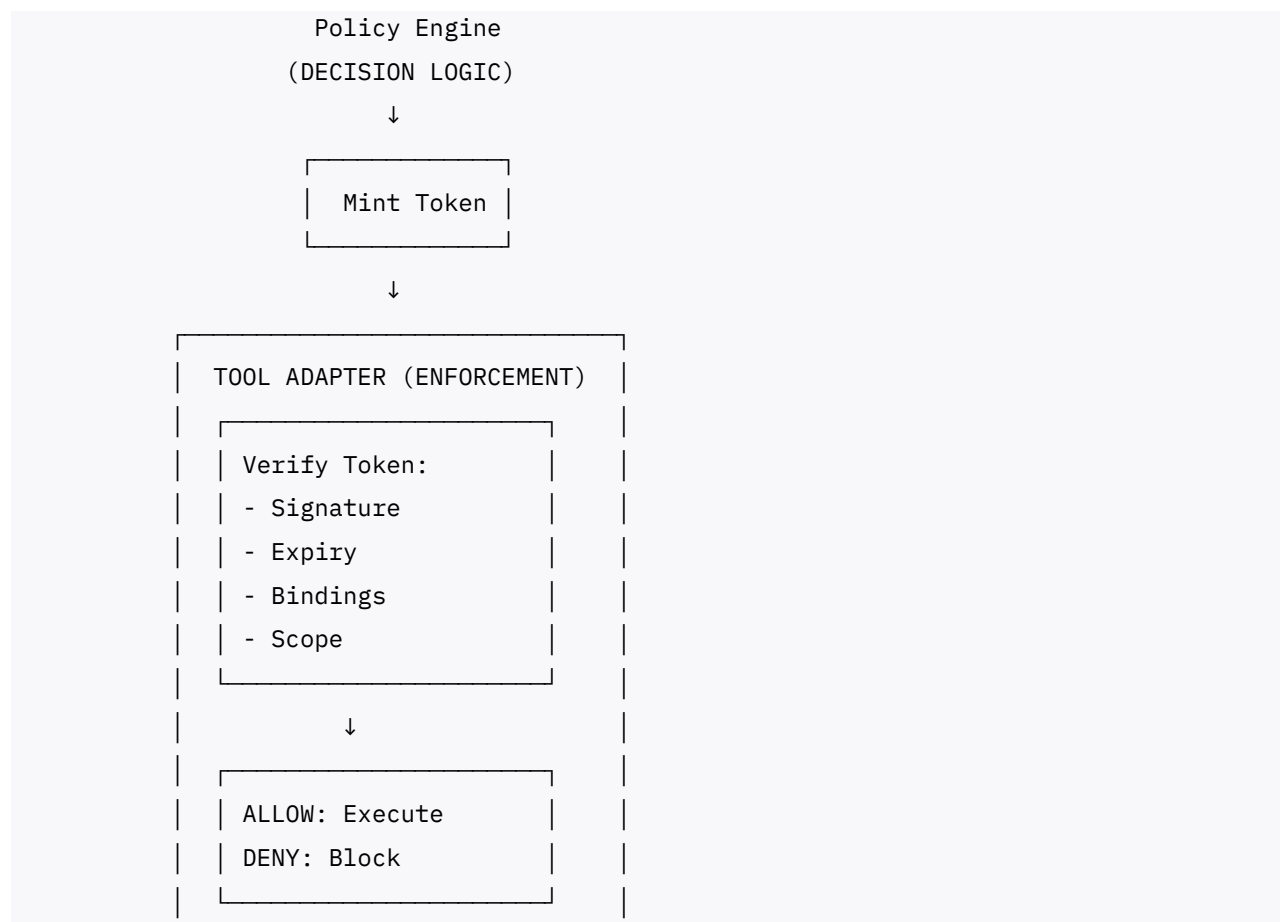
The policy engine performs these steps to issue a token:^[1]

1. Receive intent from agent
2. Canonicalize intent (RFC 8785 JSON Canonicalization)
3. Compute intent_hash = SHA256(canonical_bytes)
4. Determine risk class (LOW, MEDIUM, HIGH, CRITICAL)
 - Based on tool, action, params, subject attributes
5. Evaluate policy rules (deterministic)
 - Apply rules in priority order, first match wins

- Validate required evidence (signatures, timestamps, bindings)
 - 6. Decision: ALLOW or DENY
 - 7. If ALLOW:
 - a. Select signing key (rotation strategy: proprietary)
 - b. Generate nonce (entropy source: proprietary)
 - c. Calculate expiry (time window: proprietary)
 - d. Mint token with all binding fields
 - e. Sign token with issuer key
 - f. Return token to requesting agent
 - 8. If DENY:
 - Log decision with reason
 - Return error with required proof level (what's needed to retry)
-

7. Non-Bypassability Architecture

Capability tokens are effective ONLY when tool adapters are the exclusive execution path:^[1]



↓
TOOL (Email, Payments, etc.)

CRITICAL:

- Adapters hold exclusive credentials to tools
- No direct execution paths exist outside adapters
- Orchestrator layer cannot bypass token verification
- Even if orchestrator is compromised, tools require valid token

8. Security Properties

8.1 Non-Transferability

Tokens are bound to the requesting subject. A stolen token cannot be used by an attacker:

```
Attacker steals token (subject_id = "usr_123")
Attacker tries to use as (subject_id = "usr_456")
Adapter rejects: subject_id mismatch → UNAUTHORIZED
```

8.2 Non-Replayability (One-Time Tokens)

Tokens with `one_time: true` track nonces in a replay cache:

```
Attacker intercepts and replays token
Adapter checks: nonce_n01H already used?
Cache lookup: YES
Adapter rejects: ALREADY_USED
```

Stateless nullifiers (Poseidon hash) allow this without server state:

```
nullifier = Poseidon(token.nullifier_seed, use_index, subject_id)
Adapter verifies: nullifier not in consumed set
On use: atomically add to consumed set
Prevents reuse even across stateless adapters
```

8.3 Tamper-Evidence

Any modification to intent parameters invalidates the token:

```
Authorized: send email to alice@example.com (intent_hash = abc123...)
Attacker modifies: send email to attacker@example.com
New intent_hash = def456...
Adapter: intent_hash mismatch → TAMPERING_DETECTED → BLOCK
```

8.4 Temporal Isolation

Tokens are valid only within a time window:

```
Issued: 2026-01-08T00:00:01Z
Expires: 2026-01-08T00:05:01Z (5 minute window)
Current: 2026-01-08T00:10:00Z
Adapter: Token expired → BLOCK
```

9. Proof Levels & Evidence Binding

Tokens may require different proof levels based on risk class:^[1]

Risk Class	Proof Level	Evidence Required	Example
LOW	NONE	(none)	Reading public web content
MEDIUM	ATTESTATION	MFA, consent receipt	Sending email
HIGH	ATTESTATION	MFA + allowlist attestation	Payment transfer
CRITICAL	ZK	Zero-knowledge proof	Healthcare access

```
{
  "proof": {
    "proof_level": "ATTESTATION",
    "evidence_refs": [
      "ev_mfa_totp_123",
```

```
    "ev_user_consent_456"
  ],
  "zk_proof_ref": null
}
```

Token includes evidence reference IDs, not evidence content. Adapter optionally verifies evidence if required for that tool.

10. Comparison with Related Work

System	Purpose	Key Difference
Proof Capability Token	Cryptographic action authorization	Binds to canonical intent hash + policy hash + subject; supports parameter sealing
UCAN (Spec)	Capability delegation	Focuses on delegation chains; intent-agnostic
OAuth 2.0 Tokens	API access control	Coarse-grained scope; no action-level binding
JWT (RFC 7519)	General-purpose claims**	Self-contained; no policy binding
Macaroons	Delegated credentials	Supports caveats; not intent-bound

Key Novelty of This Protocol:

The combination of **canonical intent hashing** + **policy bundle versioning** + **temporal/subject binding** + **explicit parameter constraints** creates an authorization token that is both **verifiable** and **non-bypassable**.

11. Prior Art Licensing & Strategic Intent

This specification is published as prior art under **CC0 1.0 Universal Public Domain Dedication** to:

1. **Establish prior art** for the capability token binding pattern
2. **Prevent patent enclosure** of cryptographic action authorization methods
3. **Enable interoperability** across agent authorization platforms
4. **Block trivial patent claims** on token verification, binding, and scope enforcement

What this document covers (PUBLIC):

- Token structure and field definitions
- Binding invariants (mathematical properties)
- Verification protocol (checklist and flow)
- Parameter binding patterns
- Proof level system
- Security properties

What remains proprietary (NOT DISCLOSED):

- Risk classification algorithms (how risk class is determined)
- Policy compilation optimization (rule priority resolution)
- Nonce generation strategy (entropy source, distribution)
- Token expiry calculation (time window selection heuristics)
- Evidence validation (trust root keys, freshness scoring, shortcuts)
- Adapter load balancing and failover logic
- Replay cache implementation details

12. Implementation Guidelines

12.1 Minimum Adapter Requirements

Any implementation claiming compatibility **MUST**:

1. Verify token signature before any action
2. Check expiry with clock skew tolerance

3. Verify audience (tool + adapter_id) matches
4. Verify intent hash matches request
5. Enforce scope constraints locally
6. Implement nonce/nullifier tracking for one-time tokens
7. Emit audit events for all enforcement decisions
8. Never execute actions without valid token

12.2 Reference Verification (Pseudocode)

```
def verify_and_execute(token: CapabilityToken, intent: ActionIntent, adapter:
ToolAdapter):
    # 1. Signature
    if not verify_signature(token, issuer_pubkey):
        audit_log("DENIED: invalid_signature")
        return BLOCK

    # 2. Expiry
    if current_time > token.expires_at + SKEW:
        audit_log("DENIED: expired")
        return BLOCK

    # 3. Audience
    if token.audience.tool != adapter.tool or token.audience.adapter_id != adapter.id:
        audit_log("DENIED: wrong_audience")
        return BLOCK

    # 4. Subject
    if token.subject.subject_id != intent.subject.subject_id:
        audit_log("DENIED: unauthorized_subject")
        return BLOCK

    # 5. Intent binding
    intent_hash_computed = sha256(canonicalize(intent))
    if intent_hash_computed != token.binding.intent_hash:
        audit_log("DENIED: intent_hash_mismatch")
        return BLOCK

    # 6. Scope
```

```

if intent.action not in token.scope.actions:
    audit_log("DENIED: action_not_in_scope")
    return BLOCK

for param_name, param_value in intent.params.items():
    constraint = token.scope.constraints.get(param_name)
    if constraint and not satisfies(param_value, constraint):
        audit_log(f"DENIED: {param_name} violates constraint")
        return BLOCK

# 7. Replay protection
if token.replay_protection.one_time:
    if is_nonce_used(token.replay_protection.nonce):
        audit_log("DENIED: nonce_already_used")
        return BLOCK
    mark_nonce_used(token.replay_protection.nonce)

# 8. Budget
if token.limits.budget and spent_budget >= token.limits.budget.max_amount:
    audit_log("DENIED: budget_exhausted")
    return BLOCK

# 9. Revocation
if is_revoked(token.token_id):
    audit_log("DENIED: token_revoked")
    return BLOCK

# All checks passed
audit_log("ALLOWED")
return adapter.execute(intent)

```

13. Threat Model & Residual Risks

13.1 Threats Mitigated

- ✓ **Unauthorized action execution** – Token binding prevents execution of unapproved actions
- ✓ **Token transfer attacks** – Subject binding makes tokens non-transferable

- ✓ **Replay attacks** – Nonce/nullifier tracking prevents reuse
- ✓ **Parameter tampering** – Intent hash binding detects modifications
- ✓ **Policy bypass** – Policy bundle hash binding invalidates tokens on policy updates
- ✓ **Indefinite token use** – Temporal expiry bounds token lifetime
- ✓ **Orchestrator compromise** – Adapter-enforced verification survives orchestrator compromise

13.2 Residual Risks (Not Addressed By This Spec)

- ✗ **Adapter compromise** – If adapter itself is compromised, all bets are off
- ✗ **Key compromise** – If issuer signing keys are stolen, attacker can mint tokens
- ✗ **Policy engine manipulation** – If policy engine is compromised, may issue bad tokens
- ✗ **Side channels** – Timing attacks on signature verification not addressed
- ✗ **Signature algorithm weakness** – Depends on underlying crypto (e.g., Ed25519 resistance to quantum)

Mitigation: Use HSM/KMS for key management, compartmentalize policy engine, rotate keys regularly.

14. Revision History

Version	Date	Changes
1.0	January 8, 2026	Initial specification

End of Specification

How does this look? Would you like me to:

1. **Add more concrete examples** (e.g., worked examples for email, payments, database)?
2. **Expand the threat model** with attack scenarios and mitigations?
3. **Add EAS (Ethereum Attestation Service) schema details** for on-chain binding?
4. **Include reference implementation in TypeScript/Python?**

5. **Prepare this for arXiv submission** with formal citations and academic structure?

**

-
1. lets-discuss-licensing-open-so-HV_bbj5uQIOPuXCF7b61YQ.md
 2. Capability-Token-Semantic-Mapping.pdf
 3. Adapter Coordination Interface Specification.pdf
 4. Audit Logs_ Event Schema & Sanitized Examples.pdf
 5. LProof-Intent-Schema-Specification.pdf
 6. Conscious Sovereignty Layer (CSL) Solver.pdf
 7. Λ Proof Protocols.pdf
 8. DAO Charter.pdf
 9. Building a Λ Dapp_ A Developer's Implementation Guide.pdf
 10. Data Sovereignty Informational Personhood.pdf
 11. Λ Proof_ An Architectural Blueprint for Verifiable Artificial Intelligence.pdf
 12. Λ^P -Archivum.pdf
 13. Λ Proof - R96 Credits.pdf